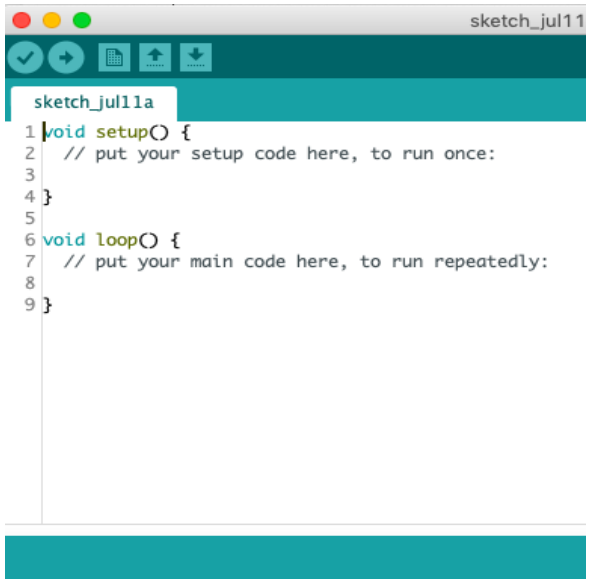
Module – 3:
Introduction To Arduino Programming

Introduction

- The Arduino Software (IDE) allows you to write programs (i.e. **sketches**) and upload them to your board.

- A sketch is consists of two **mandatory** functions:
  - ✓ **Setup( )** -- it is executed **once**
  - ✓ **Loop( )** -- it is executed **repeatedly**

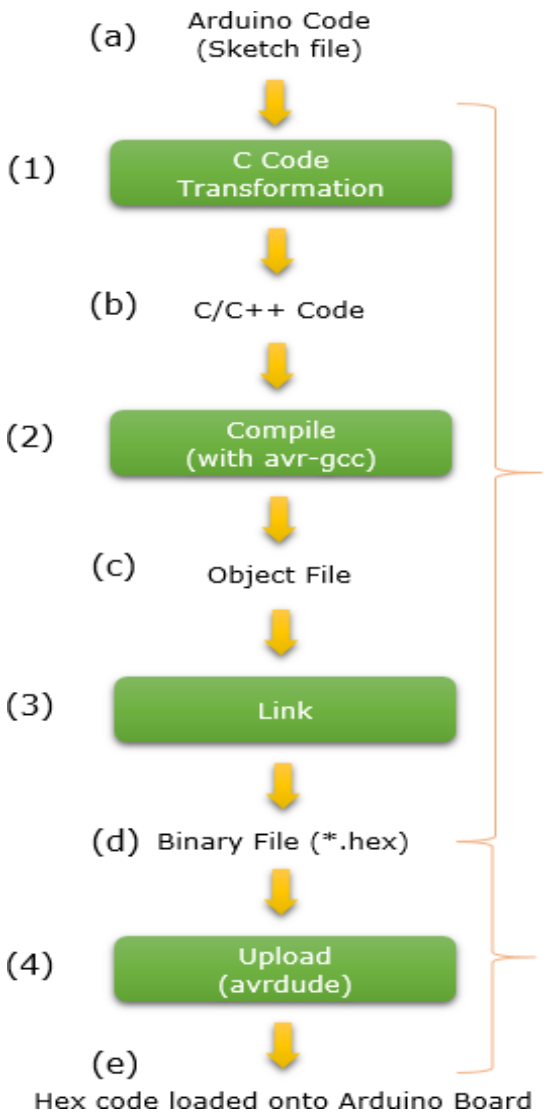


- Setup( ) is used for
  - ✓ initialization of serial communication
  - ✓ defining pinMode
  - ✓ declaring variables

- Loop( ) is used for
  - ✓ writing the main code which has to execute continuously.
  - ✓ e.g. reading inputs from the sensors, triggering outputs to the external device, etc.

Cont…

- **Sketches** are **compiled** by avr-gcc / avr-g++
  - It is based on C/C++ programming language
- So, the program syntax is almost similar to C/C++
  - Supported data types
  - Variables
  - Constants
  - Control structure
  - Looping structure
  - Arrays
  - Strings
  - Function



- One important extension is : **Arduino Libraries**
  - Libraries are a collection of code that makes it easy for you to connect to a sensor, display, module, etc.

Variables

Constants

HIGH | LOW
INPUT | OUTPUT | INPUT_PULLUP
LED_BUILTIN
true | false
Floating Point Constants
Integer Constants

Conversion

(unsigned int)
(unsigned long)
byte()
char()
float()
int()
long()
word()

Data Types

array
bool
boolean
byte
char
double
float
int
long
short
size_t
string
String()
unsigned char
unsigned int
unsigned long
void
word

Variable Scope & Qualifiers

const
scope
static
volatile

Utilities

PROGMEM
sizeof()

Operators & Structures

Sketch

loop()
setup()

Control Structure

break
continue
do...while
else
for
goto
if
return
switch...case
while

Further Syntax

#define (define)
#include (include)
/* */ (block comment)
// (single line comment)
; (semicolon)
{} (curly braces)

Arithmetic Operators

% (remainder)
* (multiplication)
+ (addition)
- (subtraction)
/ (division)
= (assignment operator)

Comparison Operators

!= (not equal to)
< (less than)
<= (less than or equal to)
== (equal to)
> (greater than)
>= (greater than or equal to)

Boolean Operators

! (logical not)
&& (logical and)
|| (logical or)

Pointer Access Operators

& (reference operator)
* (dereference operator)

Bitwise Operators

& (bitwise and)
<< (bitshift left)
>> (bitshift right)
^ (bitwise xor)
| (bitwise or)
~ (bitwise not)

Compound Operators

%= (compound remainder)
&= (compound bitwise and)
*= (compound multiplication)
++ (increment)
+= (compound addition)
-- (decrement)
-= (compound subtraction)
/= (compound division)
^= (compound bitwise xor)
|= (compound bitwise or)

Few Built-in Functions

https://www.arduino.cc/reference/en/

- **pinMode (pin, mode)**
  - It configures the specified pin to behave either as input or as output
  - By default the digital pins in Arduino function as input.
  - **pin**: is the number of the pin whose mode needs to be set
  - **mode**: can be INPUT, OUTPUT, INPUT_PULLUP.

```
pinMode(9,OUTPUT);
```

- **digitalReadPin(pin)**
  - Reads the value from a specified digital pin, either HIGH or LOW.

```
val = digitalRead(inPin);
```

- **digitalWrite(pin, value)**
  - Used for output by using the LOW/HIGH logic level (i.e. 0V / 5V)
  - **value**: LOW / HIGH

```
digitalWrite(10,HIGH);
```

- **analogRead(pin)**
  - Access and gets value from a particular Analog pin having 10-bit resolution (i.e. 10-bit ADC)
  - Returns: 0-1023 (integer)
  - Arduino UNO yields a resolution between readings of: 5 volts / 1024 units. It will map input voltages between 0 and the operating voltage(5V or 3.3V) into integer values between 0 and 1023.
  - The input range can be changed using analogReference()

```
val = analogRead(A3);
```

- **analogWrite(pin, value)**
  - Write the analog value (PWM wave) to a pin
  - **value:** it is the duty cycle value between 0 and 255 (as 6 pins).
  - Note: analogRead values go from 0 to 1023, analogWrite values from 0 to 255

```
analogWrite(9, val / 4);
```

Cont…

- **delay(ms)**
  - Pause the program for the amount of time (in millisecond) specified by **ms**

```
delay(1000); // wait for a second
```

- **Serial.begin(speed)**
  - It sets the **speed** in bps (baud rate) for serial data transmission from computer to Arduino board

```
Serial.begin(9600);
```

- **Serial.available ()**
  - Returns: the number of bytes (characters) available to read

```
if (Serial.available() > 0) { }
```

- **Serial.print(value)**
  - Print data to the serial port as human-readable ASCII text
  - Numbers are printed using ASCII character for each digit
  - Floats are printed as ASCII digits (upto 2 decimal places)
  - Bytes are send as a single character
  - Characters and Strings are sent as is.

```
Serial.print("I received: ");
```

- **Serial.print(value, format)**
  - The optional 2nd argument specifies the base (format) to use
  - **format:** BIN / OCT / DEC / HEX

```
Serial.print(i,DEC);
// Print Decimal value of number i
```

- **Serial.println(value) , Serial.println(value, format)**
  - Additionally it returns the number of bytes written

Cont…

- **Serial.read()**
  - Reads incoming serial data.

```
incomingByte = Serial.read();
```

- **Serial.write(val) or .write(str) or .write(buf, len)**
  - Writes binary data to the serial port.
  - This data is sent as a byte or series of bytes; to send the characters representing the digits of a number use the print() function instead.

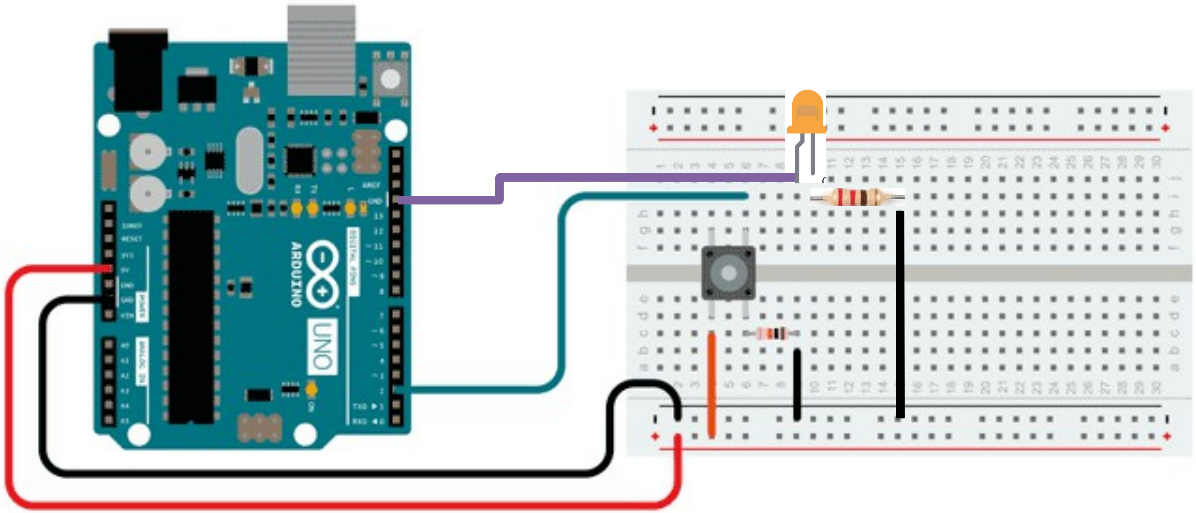
```
Serial.write(45); // send a byte with the value 45

int bytesSent = Serial.write("hello");  //send the string "hello" and return the length of the string.
```

- **Trigonometry:**
  - **cos()**
  - **sin()**
  - **tan()**

- **Math:**
  - **abs()**
  - **max()**
  - **min()**
  - **pow()**
  - **sq()**
  - **sqrt()**
  - **random()**
  - **randomSeed()**

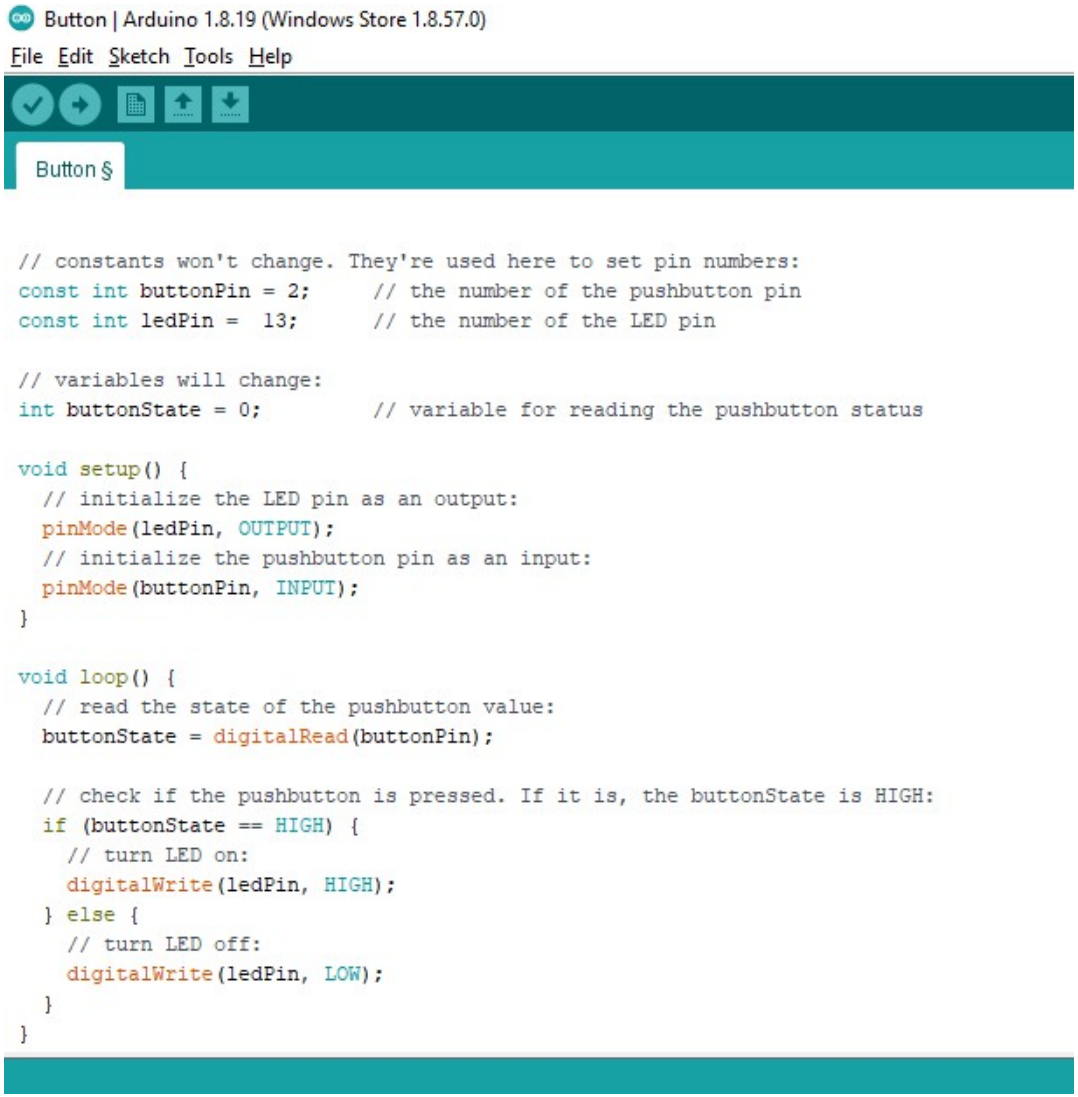
Example 1: Digital Read-Write

- Objective:
  - Turns on and off a LED connected to digital pin 13, when pressing a pushbutton attached to pin 2.



- The circuit:
  - LED attached from pin 13 to ground through 220 ohm resistor
  - One leg of the Pushbutton attached to pin 2
  - That same leg of the button connects through a pull-down resistor (here 10K ohm) to ground.
  - The other leg of the button connects to the 5 volt supply.

Cont…




```
// constants won't change. They're used here to set pin numbers:
const int buttonPin = 2;     // the number of the pushbutton pin
const int ledPin =  13;      // the number of the LED pin

// variables will change:
int buttonState = 0;         // variable for reading the pushbutton status

void setup() {
  // initialize the LED pin as an output:
  pinMode(ledPin, OUTPUT);
  // initialize the pushbutton pin as an input:
  pinMode(buttonPin, INPUT);
}

void loop() {
  // read the state of the pushbutton value:
  buttonState = digitalRead(buttonPin);

  // check if the pushbutton is pressed. If it is, the buttonState is HIGH:
  if (buttonState == HIGH) {
    // turn LED on:
    digitalWrite(ledPin, HIGH);
  } else {
    // turn LED off:
    digitalWrite(ledPin, LOW);
  }
}
```

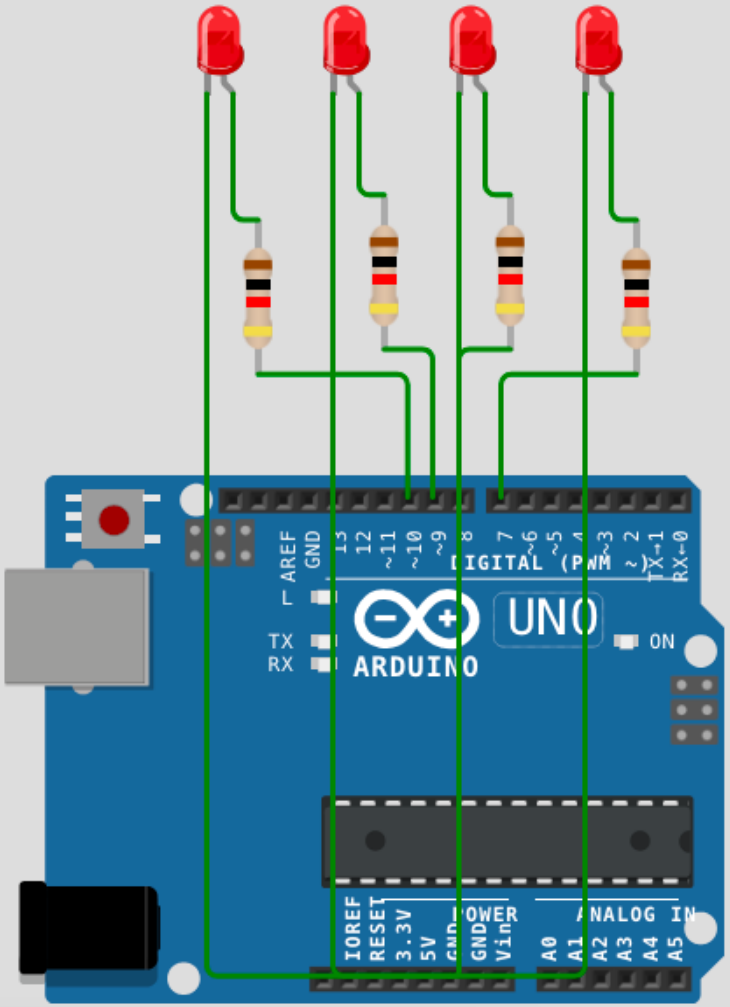
- When the pushbutton is open (**unpressed**)
  - there is no connection between the two legs of the pushbutton, so the pin is connected to ground (through the pull-down resistor) and we read a LOW.
- When the button is closed (**pressed**)
  - it makes a connection between its two legs, connecting the pin to 5 volts, so that we read a HIGH.

Example 2: Binary Counter

sketch.ino ● diagram.json ● Library Manager

```
1   int leds[] = {7, 8, 9, 10};   // LSB → MSB
2   int binary[4];
3
4   void setup() {
5     for (int i = 0; i < 4; i++) {
6       pinMode(leds[i], OUTPUT);
7     }
8   }
9
10  void loop() {
11    for (int count = 0; count < 16; count++) {
12      decimalToBinary(count);
13      displayBinary();
14      delay(1000);   // 1 second delay
15    }
16  }
17
18  // Convert decimal number to 4-bit binary
19  void decimalToBinary(int num) {
20    for (int i = 0; i < 4; i++) {
21      binary[i] = num % 2;
22      num = num / 2;
23    }
24  }
25
26  // Display binary bits on LEDs
27  void displayBinary() {
28    for (int i = 0; i < 4; i++) {
29      digitalWrite(leds[i], binary[i]);
30    }
31  }
```

Simulation



Read Analog Voltage

- ADC provide digital output which is proportional to analog value.
- To know what is input analog value, we need to convert the received digital value back to analog value through program.

Aout = digital value * (Vref/2^n – 1)

- Example:
  - digital value = 512 and ADC is 10-bit with 5V Vref.
  - What analog voltage is giving the respective digital value?

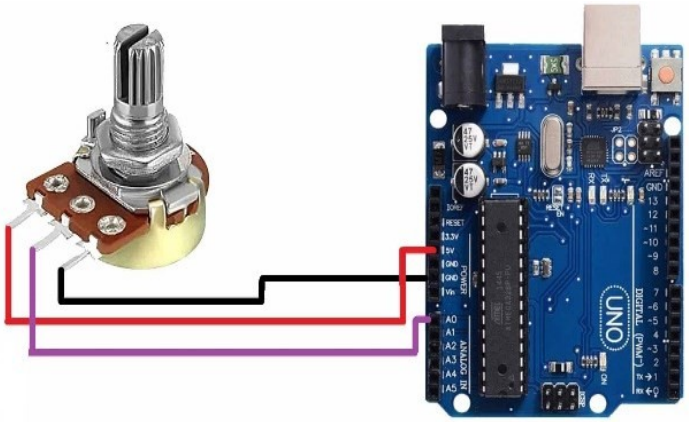
Aout = 512 * (5 V / 1023) = 2.5 V

digitalValue = analogRead (pin)

pin **-** number of analog pin which we want to read
digitalValue: 0 – 1023

Example-3: Read Analog Voltage

```
// select the input pin for the potentiometer
int sensorPin = A0;
// variable to store the value coming from the sensor
int digitalValue = 0;
float analogVoltage = 0.00;
 void setup() {
        Serial.begin(9600);
}
 void loop() {
       // read the value from the analog channel
       digitalValue = analogRead(sensorPin);
       Serial.print("digital value = ");
        //print digital value on serial monitor
       Serial.print(digitalValue);
       //convert digital value to analog voltage
       analogVoltage = (digitalValue * 5.00)/1023.00;
       Serial.print("  analog voltage = ");
       Serial.println(analogVoltage);
       delay(1000);
}
```



Pin 1 & 3 of Potentiometer: connect Vcc and GND

Pin 2 of Potentiometer: Connect with A0 pin.

COM4 (Arduino/Genuino Uno)

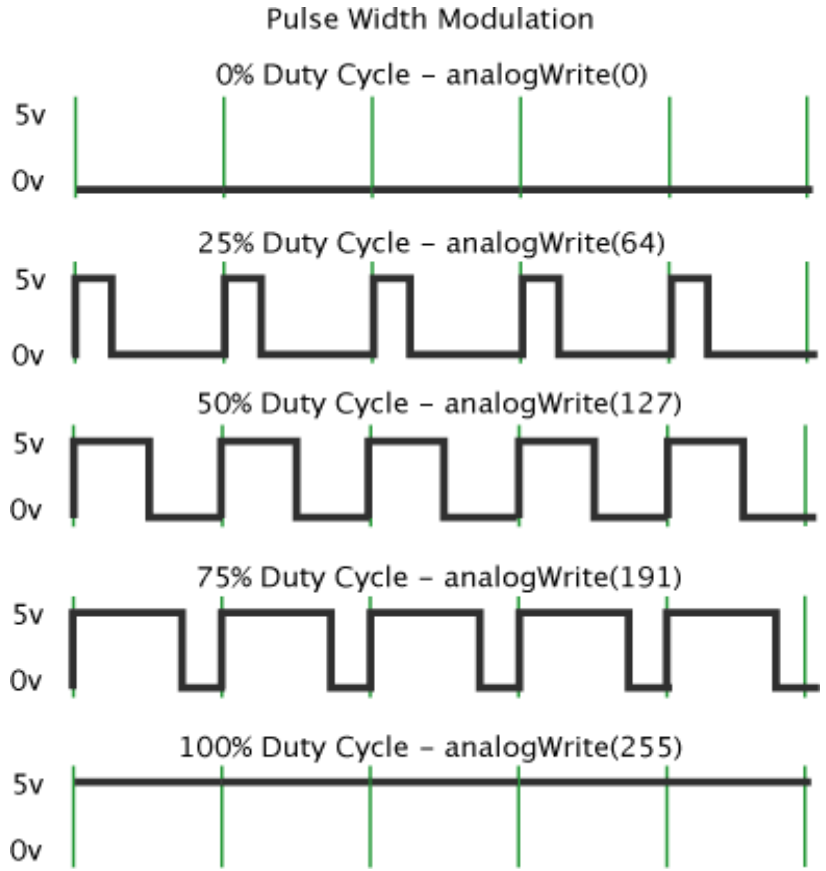
```
digital value = 0  analog voltage = 0.00
digital value = 0  analog voltage = 0.00
digital value = 30  analog voltage = 0.15
digital value = 66  analog voltage = 0.32
digital value = 171  analog voltage = 0.84
digital value = 275  analog voltage = 1.34
digital value = 331  analog voltage = 1.62
digital value = 400  analog voltage = 1.96
digital value = 459  analog voltage = 2.24
digital value = 475  analog voltage = 2.32
```

Write Analog Value

- Digital control is used to create a square wave, a signal switched between ON and OFF.

- This on-off pattern can simulate voltages in between Vcc and GND.
  - ✓ by changing the portion of the time the signal spends ON versus the time that the signal spends OFF

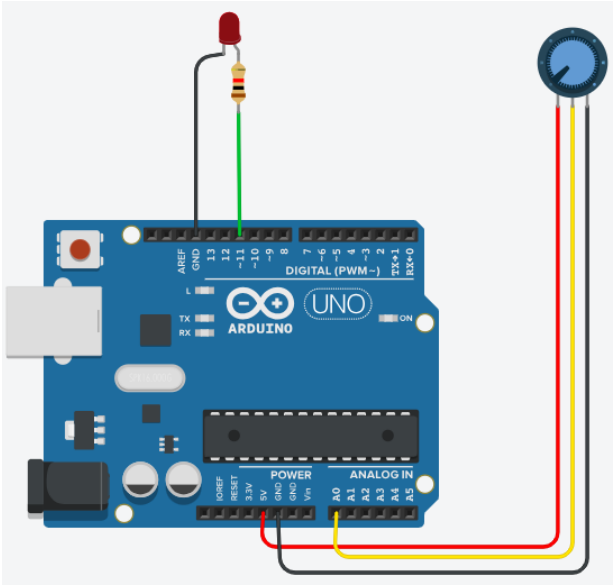
- The analogWrite(value) is on a scale of 0 – 255.
  - ✓ Zero value means 0% duty cycle, 255 value means 100% duty cycle.



Example-4: Write Analog Value

```
1  //select the input pin for the potentiometer
2  int sensorPin = A0;
3
4  //variable to store the value coming from the sensor
5  int digitalValue = 0;
6  float analogVoltage = 0.00;
7
8  //select the ledpin
9  int ledPin = 11;
10 int outputValue = 0;
11
12 void setup() {
13     Serial.begin(9600);
14 }
15
16 void loop() {
17     // read the value from the analog channel
18     digitalValue = analogRead(sensorPin);
19
20     //print digital value on serial monitor
21     Serial.print("digital value = ");
22     Serial.print(digitalValue);
23
24     //convert digital value to analog voltage
25     analogVoltage = (digitalValue * 5.00)/1023.00;
26     Serial.print(" analog voltage = ");
27     Serial.print(analogVoltage);
28
29     outputValue = map(digitalValue,0,1023,0,255);
30     analogWrite(ledPin,outputValue);
31     Serial.print(" output value = ");
32     Serial.println(outputValue);
33     delay(1000);
34 }
```

OUTPUT: LED Dimming by Potentiometer



Pin 1 & 3 of Potentiometer: connect them to Vcc and GND of Arduino

Pin 2 of Potentiometer: Connect with A0 pin of Arduino

One **LED** connected with digital pin 9 and grounded through 220 ohm or 1 Kohm resistor